Evaluating Stainless on SV-COMP

Hadi Saleh

5th May 2026

Abstract

Replace this with the actual abstract. Obviously. Suspendisse luctus leo et porta mattis. In semper, nisi et suscipit iaculis, leo urna laoreet lacus, ut laoreet lorem tellus eget dui. Vestibulum eu auctor nisi. Morbi pharetra euismod velit ac mattis. Maecenas tempor vitae augue ut aliquam. Nunc auctor, nibh at imperdiet finibus, ex leo semper lacus, ac vehicula quam nisl condimentum leo.

Contents

1 Introduction **1**
1.1 Formal Verification 1
1.2 Purpose and Goals 2
1.3 Thesis Outline 3

2 Background **5**
2.1 Stainless 5
2.1.1 Pure Scala 6
2.2 SV-COMP 6

3 Methodology **7**

4 Design **9**

5 Implementation **11**

6 Evaluation **13**

7 Conclusions **15**

List of Figures

List of Tables

Chapter 1

Introduction

As software becomes increasingly deployed to critical systems in society, everything from medical devices to vehicles to financial infrastructure. As the complexity of such systems advance, the cost of undetected defects drastically increases as well. A major important element of such software then becomes to ensure and verify that they behave correctly and as expected.

A common way to ensure this is to have them tested and verified to make sure that such software and systems do what they are intended to do, therefore making them more reliable and safe to use. Traditional testing methods, where a set of inputs gets run through the program and the outputs get checked against the corresponding expected results. While this is still useful, it would only demonstrate the presence of defects, and not their absence. For programs with large or even infinite input ranges, testing them in such a way would be infeasible.

In response to this, formal verification tools developed to provide mathematical guarantees that a program satisfies its specifications for *all* valid inputs. An example of such a tool is Stainless [1], developed at EPFL for programs written in the Scala programming language. Stainless is a deductive verification framework, meaning that it proves program correctness from formal specifications using mathematical logical reasoning. It verifies the programs functional correctness, termination and absence of runtime errors.

However, such verifiers aren't guaranteed to be flawless and can fail to verify correct programs, timeout, or produce false evaluations. To adress this issue and benchmark verification tools systematically , there is an annual verifier benchmark competition called SV-COMP [2], which maintains a large amount of benchmarks covering a wide range of verification tasks primarly focused on C and Java verifiers and validators.

Although, Stainless appeared promising in different studies (cite studies), it has yet to be evaluated on a large scale benchmark such as SV-COMP. This project addresses the gap by evaluating Stainless on SV-COMP benchmarks through translating the benchmark programs used in SV-COMP on Java verifiers, to equivalent Scala programs and running Stainless on them. The results are then evaluated and compared to the Java verifiers' benchmarks.

1.1 Formal Verification

Formal verification, the underlying concept of this project, is the process of mathematically proving that a program satisfies a set of specifications for all possible inputs.

On the other hand, testing would only be able to demonstrate the presence of bugs on a subset of inputs while formal verification provides guarantees that would hold universally. This makes formal verification particularly valuable in domains where safety is crucial and the cost of undetected defects is high.

There are multiple approaches to formal verification. **Model Checking** is the process of exhaustively exploring a program's state space to check whether a property holds or not. However, it can suffer from state space explosion for large programs [**?**]. **Static Analysis** approximates the program's behavior to infer properties without having to execute the actual program, meaning that it trades precision for scalability [**?**]. **Deductive Verification**, the approach most relevant to this project, requires that the developer annotates the program with some formal specifications to describe the expected behaviour of the program, such as preconditions and postconditions on functions, loop invariants, and assertions. The verifier then uses logical reasoning to prove that the implementation satisfies them [**?**].

SMT (Satisfiability Modulo Theories) solvers are typically relied on by deductive verifiers as they are used to solve the logical formulas that are generated from the program specifications [3]. When a verifier cannot prove a proof obligation, it passes the negation to an SMT solver. The SMT solver would in turn check the negation and if the negation is unsatisfiable, then the property holds. Otherwise, if it is satisfiable, then a counterexample can be produced from the SMT solver and returned to the verifier. SMT solvers are therefore a core part of Stainless's verification process, which is discussed further in Section 2.1.

1.2 Purpose and Goals

The purpose of this project is to evaluate Stainless on SV-COMP benchmarks which would adress the lack of systematic evaluation of Stainless. Since SV-COMP's benchmarks are written in C and Java, it won't be possible to evaluate Stainless on them directly. This project focuses solely on the Java benchmarks, for the reason being that Java and Scala are structurally similar which would make the translation process manageable. A representative subset of those Java benchmarks will be translated to Scala programs that Stainless can process. The subset chosen has a feasible amount of benchmarks for the time period of this project and excludes all the benchmarks that are not translatable to Scala programs that would be compatible with Stainless. Such untranslateable programs include:

- Programs that use Java specific features that are not supported by Stainless
- Programs that use external Java packages that are not available in Stainless

The translated benchmarks are then run on Stainless and the outcomes are compared against both the expected verdicts of the benchmarks which every SV-COMP benchmark has and against the results for the Java verifiers. The evaluation considers the verification outcomes (valid, invalid, timeout or unknown) as well as the verification time and CPU usage.

This project aims to establish a clearer picture of where Stainless stands relative to other verification tools on a standardized set of benchmarks. As well as to identify which types of programs Stainless handles well and which types of programs it struggles with.

1.3 Thesis Outline

Chapter 2

Background

This chapter goes over the foundational concepts and tools necessary to be able to understand this project, the work being done, and the results. I will start by going over formal verification in general, then go over Scala and Stainless and finally go over SV-COMP and its benchmarks and infrastructure.

2.1 Stainless

In this section, I will give an overview of Stainless, the deductive verification framework used in this project. I will describe how it works, its verification process and the subset of Scala it supports, known as Pure Scala.

Stainless [1] is a deductive verifier used to prove correctness properties on Scala programs. Stainless takes Scala programs annotated with specifications such as pre- and postconditions and attempts to prove their correctness using SMT solvers. Properties that Stainless verifies include, termination, absence of runtime errors and functional correctness.

Users annotate the Scala programs with specifications and preconditions such as `require` and postconditions such as `ensuring`. Stainless will then attempt to prove that the specifications hold for all the possible inputs of the functions, using SMT solvers rather than testing. For each condition, Stainless reports one of four verdicts, `valid`, `invalid` (with a counterexample), `timeout` or `unknown`.

The `add` function from Listing 2.1 is a simple example of a Scala function that is annotated with preconditions using `require` and postconditions using `ensuring`. The `require` condition is checked once the function is called whilst the `ensuring` condition is checked once the function has terminated. In this example, if the postcondition is violated for any input values that satisfy the precondition, then Stainless will report an invalid verdict with a counterexample. With this specific example, the postcondition will always be met if the precondition is met, therefore it would be reported as valid by Stainless.

```
1  def add(x: BigInt, y: BigInt): BigInt = {
2    require(x > 0 && y > 0)
3    x + y
4  }.ensuring(res => res > x && res > y)
```

Listing 2.1: A simple annotated Scala function verified by Stainless.

Stainless is however not compatible with all Scala programs. It only supports a subset of Scala known as Pure Scala.

2.1.1 Pure Scala

Pure Scala is a purely functional subset of Scala that Stainless accepts as input [4]. Understanding it is directly relevant to this project, as it is a key factor in determining which SV-COMP Java benchmarks are translatable into programs that Stainless can process.

Pure Scala is structured around ADTs (algebraic data types), programs are built from `abstract class` hierarchies and immutable `case class` definitions [4]. Pure Scala does support methods, inheritance, and overriding, but restricts classes to immutable fields. However, mutable `var` fields in classes and null references are not supported [5]. Mutable data structures from the Scala standard library are not supported, as Pure Scala enforces a functional, side-effect-free style, which means that functions cannot modify a state outside of their own scope [4].

For numeric types, Pure Scala provides `BigInt` for unbounded integers with no overflow semantics and `Int` as a 32-bit integer with standard overflow semantics. In this project, `BigInt` is preferred over `Int` when translating Java programs that use Java's `int` type, as it avoids overflow cases which complicates the verification process.

Beyond Pure Scala, Stainless also supports an extension that makes it possible to use `var` declarations and `while` loops [6]. Stainless handles them by transforming them into equivalent Pure Scala constructs before verification. This is directly relevant for this project as Java `for` loops are converted into `while` loops in corresponding translated Scala programs. `while` loops can be annotated with loop invariants which Stainless verifies as holding on each iteration of the loop and on termination of the loop.

2.2 SV-COMP

Chapter 3

Methodology

Chapter 4

Design

Chapter 5

Implementation

Chapter 6

Evaluation

Chapter 7

Conclusions

Bibliography

[1] EPFL LARA. Stainless Formal Verification for Scala; 2025. https://epfl-lara.github.io/stainless/.

[2] Beyer D. Competition on Software Verification (SV-COMP); 2026. https://sv-comp.sosy-lab.org/.

[3] Barrett C, Fontaine P, Tinelli C. The Satisfiability Modulo Theories Library (SMT-LIB); 2016. https://smt-lib.org/.

[4] EPFL LARA. Pure Scala Stainless Documentation; 2024. Accessed: 2025. https://epfl-lara.github.io/stainless/purescala.html.

[5] Kuncak V, Hamza J. Stainless Verification System Tutorial. In: Formal Methods in Computer Aided Design, FMCAD 2021, New Haven, CT, USA, October 19-22, 2021. IEEE; 2021. p. 2-7. Available from: https://doi.org/10.34727/2021/isbn.978-3-85448-046-4_2.

[6] EPFL LARA. Imperative and Other Effects — Stainless Documentation; 2024. Accessed: 2025. https://epfl-lara.github.io/stainless/imperative.html.